


# Machine Learning

Università degli Studi di Verona
Imbriani Paolo, VR553697
paolo.imbriani@studenti.univr.it
Professor Cigdem Beyan

Anno Accademico 2026/2027

# Indice

1 Classifiers ..... 3
1.1 Bayes classifiers ..... 4
1.1.1 Problems ..... 6
1.1.2 Solutions (one of many) ..... 6
1.2 MLE in Naive Bayes (discrete case) ..... 7
1.2.1 Continous Features: Guassian MLE ..... 7
1.3 Nonparametric classifiers ..... 7
1.4 Non-parametric techniques ..... 8
1.5 K-Nearest Neighbour ..... 10
1.5.1 Pros and Cons ..... 12

# 1 Classifiers

A fundamental **statistical approach** for classification. Hypothesis:

- The decsiion problem is cast in *probabilistic terms*;
- All relevant **probabilities are known**.

Goal:

- Discriminate the different decision rules using the probabilities and associated costs
- Estimating the **joint probability p(x,y)** from the training dataset.

**Example**

Let $y$ is what we want to classify and it is to be probabilistically described. There are two classes $y_1$ and $y_2$ which correspond to two possible states:

- P(y=y_1) = 0.7
- P(y=y_2) = 0.3

These are called **prior probabilities** and they are known. There are no measuremetns or observations available to help make a decision about which class **y** belong to. How do we decide?

- Decide $y_1$ if $P(y_1) > P(y_2)$, otherwise decide $y_2$.
- Given the probabilities (70% for $y_1$ and 30% for $y_2$ ), the decision rule would lead to classifying y as $y_1$ since 0.7 is greater than 0.3

**Example**

Having the previous Hypothesis and additionally a single measurement $x$, which is a random variable that depends on the class $y_j$, we can get:

$$p(x|y_j)_{j} = 1, 2 = \text{Likelihood}$$

i.e. the probability of having the measurement $x$ given the class $y_j$. When we fix the measurement $x$, the higher the likelihood $p(x|y_j)$, the more likely it is that the measurement $x$ belongs to class $y_j$.

## 1.1 Bayes classifiers

*Bayes formula* is a formula that makes us able to compute the posterior probability $p(y_j|x)$, i.e. the probability of having the class $y_j$ given the measurement $x$.

$$p(y_j \mid x) = P(y_j|x)p(x) = p(x|y_j)P(y_j)$$

that is

$$P(y_j \mid x) = \frac{p(x|y_j)P(y_j)}{p(x)} \iff \text{posterior} = \frac{\text{likelihood} * \text{prior}}{\text{evidence}}$$

The posterior is the probability that $y$ is $y_j$ given the observation $x$. The most important factor is the product likelihood x prior, the evidence $p(x)$ is simply a scale factor, which ensures that

$$\sum_j P(y_j|x) = 1$$

From the formula of Bayes derives the **Bayes's decision rule**: Decide $y_j$ if $P(y_j|x) > P(y_k|x)$, $y_k$ otherwise.

There's also **Naive Bayes Classifier**, a probabilistic based on **Bayes Theorem**. It assumes *independence* among features. It calculates the posterior probability using:

$$P(y_j|x) = \frac{p(x|y_j)P(y_j)}{p(x)}$$

Assume features $x = (x_1, x_2, ..., x_n)$ are independent given the class label $\omega$:

$$P(x|y) = P(x_1, x_2, ..., x_n \mid y) = \prod_{i=1}^{n} P(x_i|y)$$

*The probability of each class is determined from the training dataset.*

Example

We want to classify an email as either:

- spam
- not spam

We'll use a few features (words) that appears in the email.

| Email | Contains "Free"? | Contains "Win"? | Class |
|---|---|---|---|
| E1 | Yes | Yes | Spam |
| E2 | Yes | No | Spam |
| E3 | No | Yes | Spam |
| E4 | No | No | Ham |
| E5 | No | No | Ham |

1. Compute class priors:

$$P(\text{spam}) = \frac{3}{5}$$

$$P(\text{Ham}) = \frac{2}{5}$$

2. For each feature (word) and class, we estimate how often the word appears when the email belongs to that class.

$$P(\text{free} \mid \text{spam}) = \frac{2}{3}$$

$$P(\text{free} \mid \text{ham}) = \frac{0}{2}$$

$$P(\text{money} \mid \text{spam}) = \frac{1}{3}$$

$$P(\text{money} \mid \text{ham}) = \frac{0}{2}$$

1. Should be calcuation of evidence but in practice, for classification. $P(x)$ is the same for all classes, it's just a normalization constant. Therefore we can skip its calculation to estimate the posterior probability.

Let's classify a new email: it contains «free», does **not** contain «Win». We compute (ignoring the denominator (evidence) since its the same for both classes):

$$P(\text{free} \mid \text{Free, no win}) \leq P(\text{spam}) \cdot P(\text{free} \mid \text{spam}) \cdot P(\text{money} \mid \text{spam}) = \frac{3}{5} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{2}{15}$$

$$P(\text{ham} \mid \text{Free, no win}) \leq P(\text{ham}) \cdot P(\text{free} \mid \text{ham}) \cdot P(\text{money} \mid \text{ham}) = \frac{2}{5} \cdot \frac{0}{2} \cdot \frac{0}{2} = 0$$

Classify an observation $x$ to classify $y_j$ if:

$$P(y_j \mid x) = \frac{p(x|y_j)P(y_j)}{p(x)} \text{ is maximised}$$

- Advanteges:
  - Fast to train and predict, suitable for large datasets,
  - Works well with high dimensional data,
- Limitations:
  - Independecne assumption may not hold in real world scenarios, leading to suboptimal performance,
  - Zero probability problem: requires techniques for unseen features-class combinations.

### 1.1.1 Problems

To create an **optimal classifier** that uses the Bayesian decision rule, you need to know:

- **Prior probabilites** P(y_i)
- **Class conditional densities** (likelihood) p(x|y_i)

The performance of a classifier *strongly* depends on the **goodness** of these components. But practally all this is information is not available!

More often we only have:

- A vague knowledge of the problem, from which to extract vague **a-priori** probabilities P(y_i)
- Some particularly representative patterns, training data, used to train the classifier (often too few or sparse!)

### 1.1.2 Solutions (one of many)

We approximate it by **fitting a parametric model** (e.g Gaussian) to the training data and estiamtes its parameters using e.g., **Maximum Likelihood Estimation** (MLE). It's a standard way to estimate the parameters of a probability distribution given a set of observations.

$$\theta_j = (\mu_j, \sigma_j)$$

$$P(x|y_j) \approx N(\mu_j, \sigma_j)$$

MLE chooses the parameters values that make the observed data **the most probable**.

## 1.2 MLE in Naive Bayes (discrete case)

Suppose you're training a Naive Bayes classifier with categorical features. For each class $y$ you estimate:

- The prior $P(y)$ and the likelihood $P(x|y)$ using MLE.
  1. Prior via MLE:

$$P(y) = \frac{\text{number of samples with class y}}{\text{total number of samples}}$$

  1. Likelihood via MLE:

$$P(x_i = v_k|y) = \frac{\text{count of (x_i = v_k and y)}}{\text{count of (y)}}$$

### 1.2.1 Continous Features: Guassian MLE

If features are continuous and you assume a Guassian likelihood:

$$P(x_i|y) = \mathcal{N}(x_i, \mu_{iy}, \sigma_{iy}) \longrightarrow P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_{iy}^2}} \exp\left(-\frac{(x_i - \mu_{iy})^2}{2\sigma_{iy}^2}\right)$$

Then MLE gives:

$$\hat{\mu}_{iy} = \frac{1}{N}\sum_{j=1}^{N} x_{ij}$$

$$\hat{\sigma}_{iy}^2 = \frac{1}{N}\sum_{j=1}^{N} (x_{ij} - \hat{\mu}_{iy})^2$$

These are just **sample mean and variance** of that features within each class.

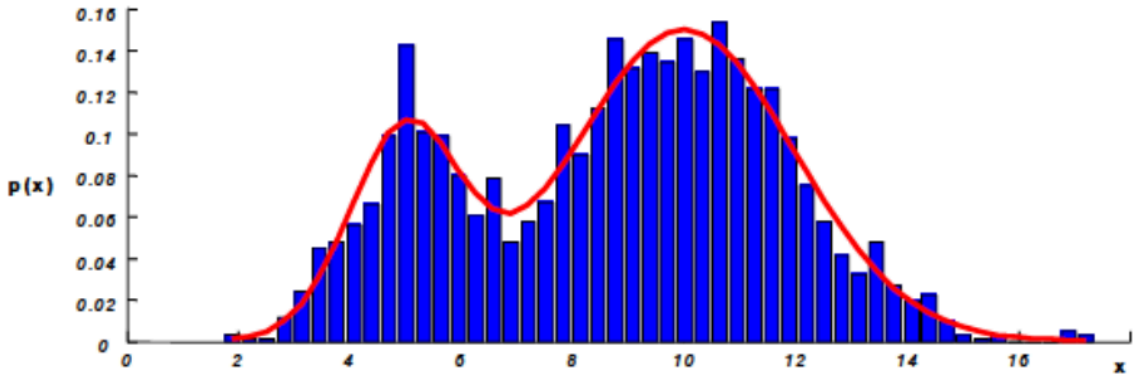
**Example**

Completa a casa

There's also other solutions for estimating the class conditional densities, such as **non-parametric methods**.

## 1.3 Nonparametric classifiers

Given a set of examples, model the density function of the data without makig any assumptions about the *form of distribution*. Estimate the probability density function *directly from the samples*. The simplest form of non-parametric density estimation is the **histogram**.




Dividing the sample space into a number of bins and approximate the density at the center of each bin by the fraction of points in the training that fall into that bin.

$$p_{h(x)} = \frac{1}{N}\frac{[\#\text{ of x}\widehat{}\text{(k in the same bin as x)}]}{[\text{bin width}]}$$

The histogram requires two parameters to be defined: bin width and starting position of the first bin. There are few problems with histograms:

- **The density** is not smooth, it **is piecewise constant**, and the estimate is discontinuous at the bin edges.
- The **discontinuities** of the estimate are not due to the underlying density; they are only an artifact of the chosen bin locations
- A much more serious problem is the **curse of dimensionality**, since the number of bins grows exponentially with the number of dimensions.
  - In high dimensions we would require a very large number of examples or else most of the bins would be empty
  - As the number of features or dimensions in a dataset increases, the amount of data required to effectively cover the feature space also increases exponentially.
  - To mitigate the curse of dimensionality, various techniques such as **feature selection**, **dimensionality reduction** (e.g., principal component analysis), and **regularization methods** are employed to reduce the dimensionality of the data while preserving its important characteristics.

## 1.4 Non-parametric techniques

Before we proceed any futher let us return to the basic defintion of probability to get a solid idea of what we are trying to accomplish:

- The probability that a vector $x$ drawn from a distribution $P(x)$ will fall in a region $\mathcal{R}$ of the sample space is:

  $$P = \int_{x' \in \mathcal{R}} P(x')\ \text{dx'}$$

  where $x$ is a particular vector we are asking about and $x'$ is a dummy variable used to sum/ integrate over all vectors in $\mathcal{R}$.

- Suppose now that $N$ vectors $\{x(1), ..., x(n)\}$ are drawn independently and identically distributed from the distribution. The probability that $k$ of these $N$ vectors fall in $\mathcal{R}$ is given by the binomial distribution:

$$P(k \in \mathcal{R}) = \binom{N}{k} P^k (1-P)^{N-k}$$

  where $\binom{N}{k} = \frac{N!}{k!(N-k)!}$ is the binomial coefficient. and the expected value for $k$ is $E[k] = \text{NP}$ The expectation $E$ is the average value we would get if we repeated drawing samples many times.
- The mean and variance of the ration $\frac{k}{N}$ are

$$E\left[\frac{k}{N}\right] = P \text{ and } \text{Var}\left[\frac{k}{N}\right] = E\left[\left(\frac{k}{N} - P\right)^2\right] = \frac{P(1-P)}{N}$$

  Therefore as $N$ increases, the variance of $\frac{k}{N}$ decreases and the estimate becomes more accurate.

$$P \approx \frac{k}{N}$$

  On the other hand, if we assume that $\mathcal{R}$ is *so small* that $P(x)$ does not vary appreciably within it, then:

$$\int_{\mathcal{R}} P(x')dx' \approx P(x)V$$

  where $V$ is the volume of the region $\mathcal{R}$.
- The general expression for **Nonparametric density estimation** is then:

$$p(x) \approx \frac{k}{NV}$$

  When applying this result to practical density estimation problems, two basic approaches can be adopted:
  - Fix $V$ and determine $k$ from the data. This leads to *kernel density estimation (KDE)*, e.g. Parzen window
  - Fix $k$ and determine $V$ from the data. This given rise to the **k-nearest neighbor** method.

  Both $k - NN$ and KDE converge to the true probability density as $N \to \infty$, provided that $V$ shrinks with $N$, and that $k$ grows with $N$ appropriately.

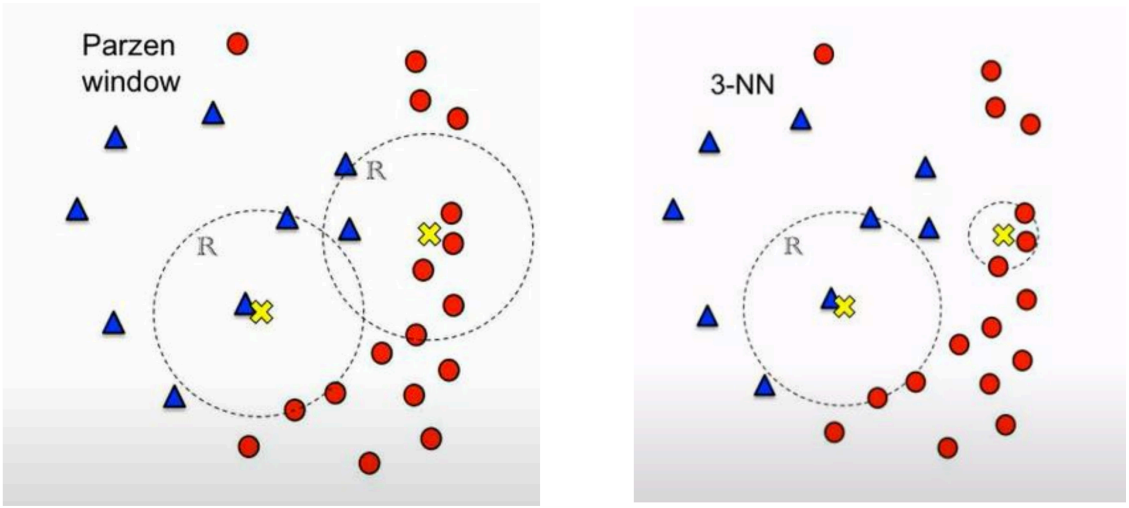



Figura 1: Parzen vs k-NN

## 1.5 K-Nearest Neighbour

Given an unknown point $x^*$, we consider $s_i \in \{s_1, ..., s_N\} = S$ the nearest point (NN) of the point $x^*$ if

$$d(x^*, s_i) = \min_l d(x^*, s_l), l = 1, ..., N$$

where $d(.)$ is distance measure defined on the feature space. If you interpret each point of a set $S$ as a prototype of a class, you can then see the equation above as a rule of classification $1 - \mathbb{N}$ associating the sample $x$ to the class $j$ to which the point $s_i$ belongs. To calculate the distance, we usually use the **Euclidean Distance**.

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

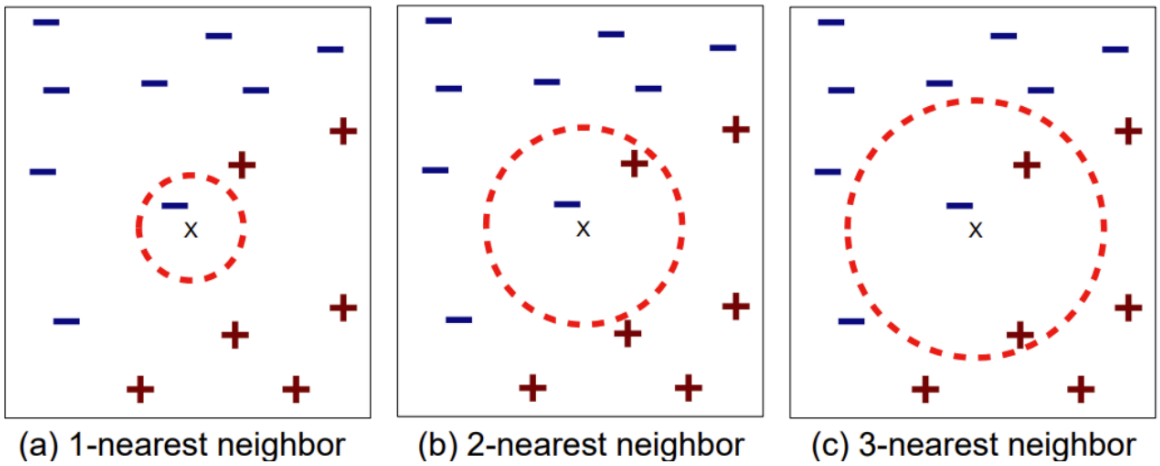



Figura 2: We choose the class using the majority vote of the k nearest neighbors, in (a) and (b) we chose blue class, in (c) we chose red class because of the majority of the neighbours.

All the closest neighbors has the same **weights**, meaning that they are equally imoportant to make a decision. Therefore the algorithm is a very sensitive to the number **k**. This can be by weighing the contribution of neighbors based on distance $w = \frac{1}{d(x^*, s_j)}$, such that the closes neighbour would have more importance.

The coice of $k$ is important because:

- If $k$ is too small, the approach is sensible to noise
- If $k$ is too large, the neighborhood may include points from other classes, leading to misclassification.

To operate correctly the attributes must have the same value scale and must therefore be *normalized* in the preprocessing phase. Normalizing the scale may not b sufficient when there are different data distributions. The mahalanobis distance is a distance measure that takes into account the correlations of the data set and is scale-invariant. It is defined as:

$$D = \sqrt{(x-\mu)^T S^{-1}(x-\mu)}$$

where $x$ is the vector of the observation, $\mu$ is the mean vector of the distribution and $S$ is the covariance matrix.

When to use **mahalanobis distance** instead of euclidean distance?

- **Correlated Features**: changes in one features are associated with changes in another
- **Unequal Variances**: MD normalized the distance based on the variances of the features, preventing features with larger variances from dominating the distance calculation.
- **Outlier detection**: Outliers often have a large MD from the rest of the data points, especially if they deviate from the normal covariance pattern.
- In **high dimensional spaces**, ED can be problematic (curse of dimensionality) incorpating information about the covarinance structure, can mitigate this issue to some extent.

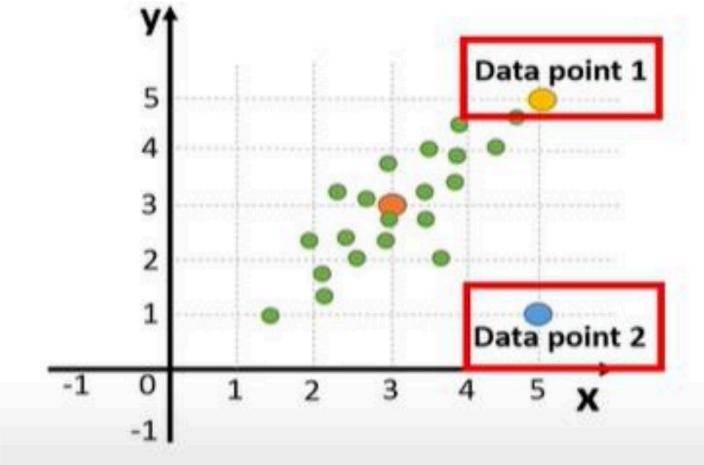



Figura 3: Euclidean distance

$$\text{CENTROID} = \begin{pmatrix} \hat{X} \\ \overline{Y} \end{pmatrix} = \begin{pmatrix} 3.1 \\ 3.0 \end{pmatrix}$$

Euclidean distance between the centroid and the point 1 is:

$$d = \sqrt{(5-3.1)^2 + (3-3.0)^2} = 2.76$$

Euclidean distance between the centroid and data point 2:

$$d = \sqrt{(2-3.1)^2 + (4-3.0)^2} = 2.76$$

These means that both points are equally distant from the centroid, but they are not equally important to the classification.

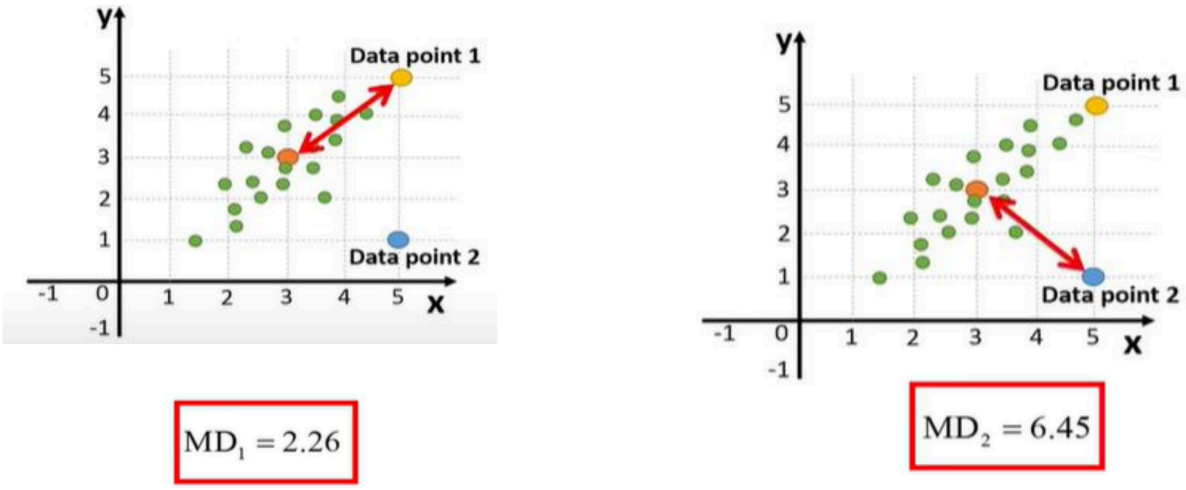


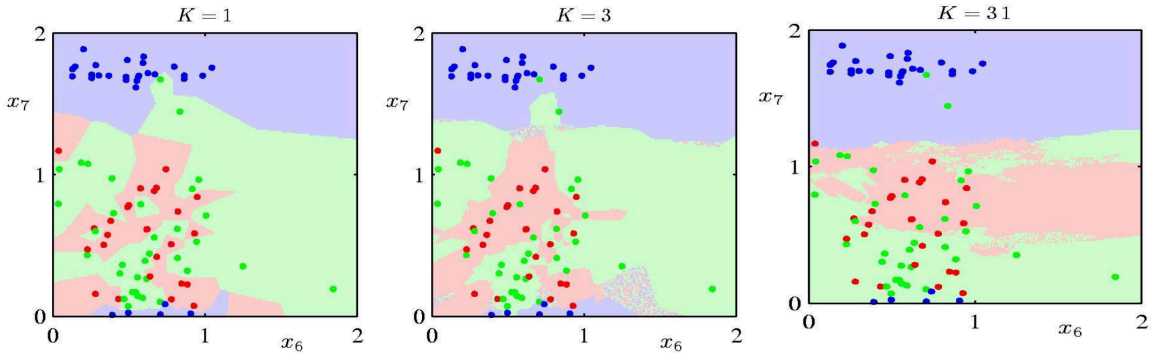

Figura 4: Mahanobis distance

Mahanobis distance takes in account correlation data and since the data is very correlated the data point 1 is more important to the classification than data point 2.

How to select $k$ in $k - \mathbb{N}$ then? Common heuristics are:

- Often 3,5,7
- Choose an odd number to avoid ties

Use your training and validation data to decide $k$.




What is the role of $k$?
How does it relate to overfitting and underfitting?

Figura 5: We can see how the accuracy changes with different values of k. The best value of k is the one that gives the highest accuracy on the validation set. And with the K you can either overfit or underfit

If there are undetermined cases (ties) there are some alternatives:

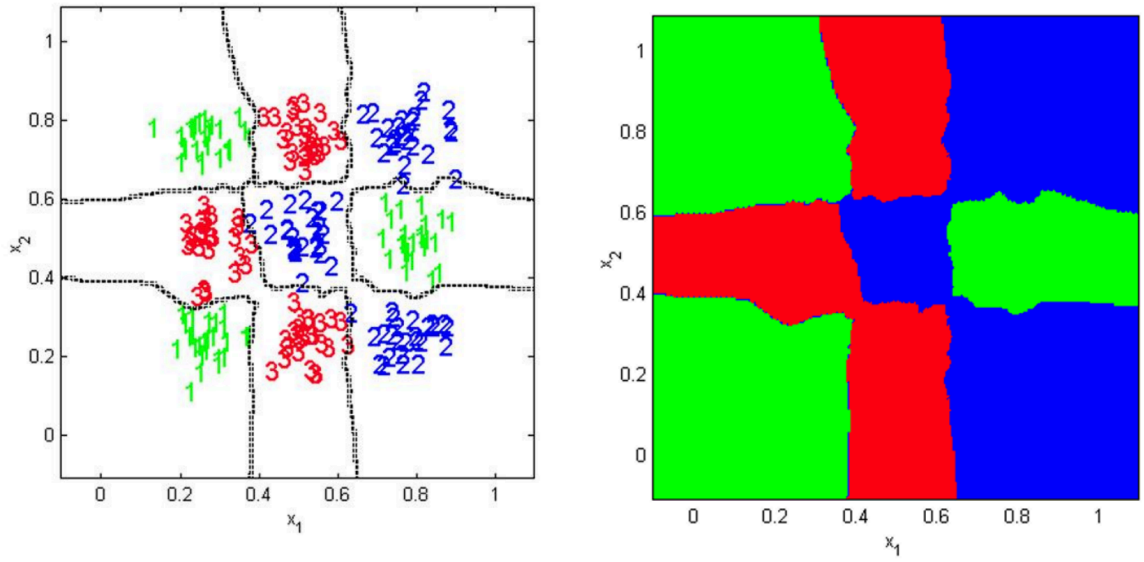
- Arbitrary choice
- Assign $x$ to the class that has the nearest average sample
- Assign $x$ to the class that has the $k_i$ samples with the least distance from it

### 1.5.1 Pros and Cons

The pros of K-NN are:

- Almost no assumptions about the data
- Non-parametric
- There is no training phase

- Allows us to construct non-linear class boundaries and are therefore more flexible.




The cons of K-NN are:

- Need to handle missing data,
- Need to be aware to overfit or underfit with the $K$ and also the distance metric must be chosen carefully,
- Sensitive to class outliers and many irrelevant attributes,
- Computationally expensive:
  - Poor scalability, we need to perform as many comparisons as labaled data.
  - You need to store all data points, so it's better when the number of samples $N$ is low.

Going back to PDF (probability density function) estimation:

- Instead of fixing V as in case for kernels, we fix the value of $k$ and enlarge

the radius of the sphere to include $k$ samples.

- Let's see the priori probabilty:
  - Given a class $y_j$, the frequency of occurence of samples $N_j$ of class $j$

  is generally simply measured in relation to the total number of samples $N$, i.e:

$$P(y_j) \equiv \hat{p}(y_j) = \frac{N_j}{N}$$

  - 2 classes $y_i$ and $y_j$ containing $N_i$ and $N_j$ samples, respectively. $N = N_i + N_j$.
  - The local density estimation for $y_i$ is calculated as (and for similarly for $y_j$)

$$\hat{p}(x|y_j) = \frac{1}{V}\frac{k_i}{N_i}$$

  such that the ratio of $k_i(k_j)$ points to the total $N_i(N_j)$ belonging to class $y_i(y_j)$, and the volume $V$: The Bayes rule says $p(x|y_j)P(y_i) > p(x|y_j)P(y_j)$ then

$$\hat{p}(x|y_i)P(y_i) > \hat{p}(x|y_j)P(y_j) \longrightarrow \frac{1}{V}\frac{k_i}{N_i} \cdot \frac{N_i}{N} > \frac{1}{V}\frac{k_j}{N_j} \cdot \frac{N_j}{N}$$
$$\longrightarrow k_i > k_j$$

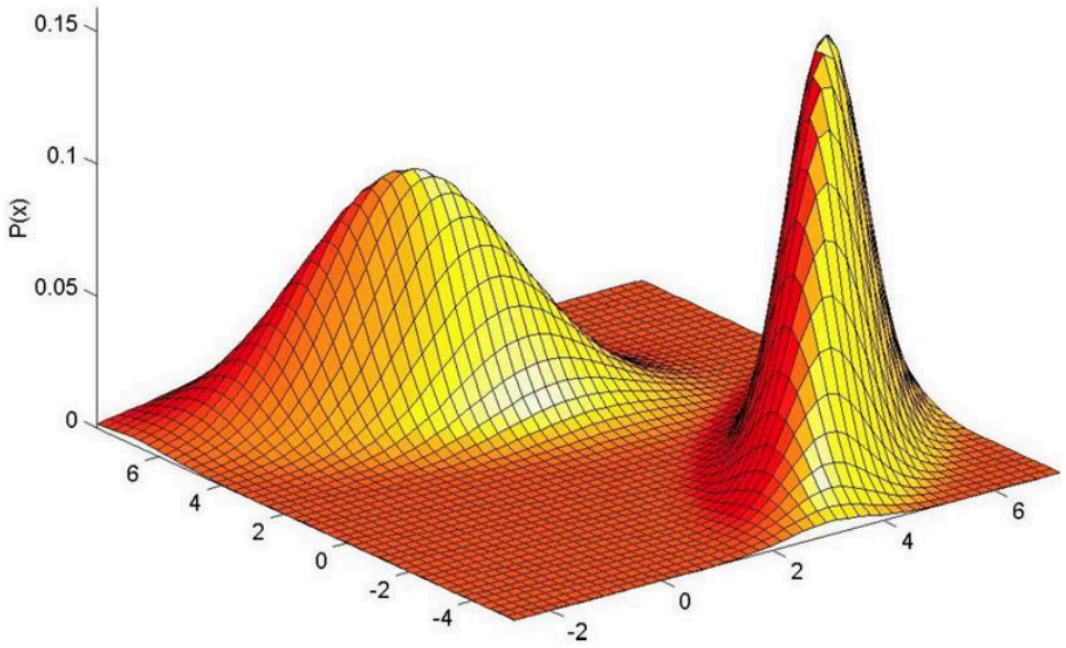



Figura 7: The true density, a mixture of bivariate Gaussians

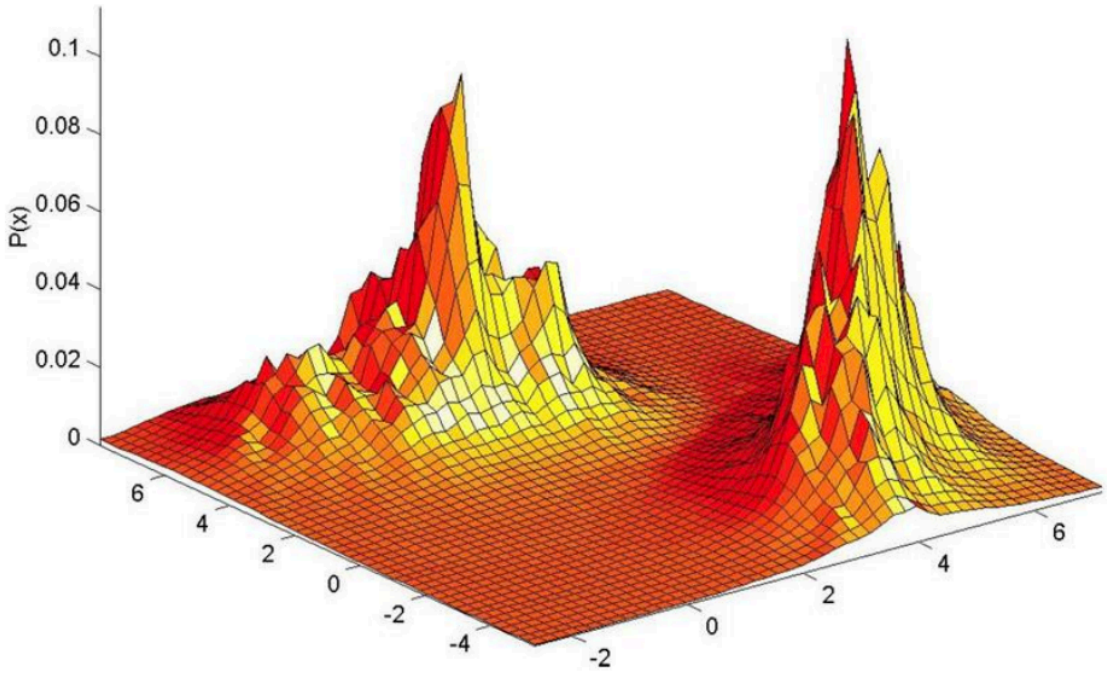



Figura 8: Density estimate for k = 10 neighbous and N = 200 examples

Notice that the estimate can be «jagged» and discontinuities in the slopes generally occur along lines away from the positions of the points themselves.